

```
<name>dfs.replication</name>
<value>2</value>
</property>
```

Restart the Hadoop services. Increase the replication for existing files and explore <http://h-mstr:50070/>.

```
$ hadoop fs -setrep -R 2 /user/fedora/
```

These experiments illustrate the resilience and flexibility of the Hadoop distributed file system, and the implicit optimisation to minimise the transfer of data across the network. You don't need to worry about distribution of data.

During your experiments, in case you come across the error stating that the *namenode* is in safe mode, run the following command:

```
$ sudo runuser hdfs -s /bin/bash /bin/bash -c "hdfs dfsadmin
-safemode leave"
```

Loading data programmatically

It is likely that you will need to get the data from some existing sources like files and databases and put what you need into HDFS. As the next experiment, you can take up the document files you may have collected over the years. Convert each into text and load them. Hadoop works best with large files and not with a collection of small files. So, all files will go into a single HDFS file, where each file will be a single record with the name of the file as the prefix. You don't need to create a local file. The output of the program can be piped to the *hdfs dfs -put* command.

The following Python program *load_doc_files.py* illustrates the idea.

- It selects document files.
- Each document file is converted to a text file.
- The name of the file is output with a delimiter (a tab, in this case).
- Each line from the file is read and output with the new line character replaced by a delimiter.
- On reaching the end of the file, a new line character is written to the standard output.

```
#!/usr/bin/python
from __future__ import print_function
import sys
import os
import subprocess

DELIM='\t'
FILETYPES=['odt','doc','sxw','abw']
# use soffice to convert a document to text file
def convert_to_text(inpath,infile):
    subprocess.call(['soffice','--headless','--convert-
to','txt:Text',
```

```
'--outdir','/tmp','/'.join([inpath,infile])])
    return '/tmp/' + infile.rsplit('.',1)[0] + '.txt'
# Convert & print file as a single line replacing '\n' by '\t'
adding file name at the start
def process_file(p,f):
    print("%s"%f, end=DELIM)
    textfile = convert_to_text(p,f)
    for line in open(textfile):
        print("%s"%line.strip(),end=DELIM)
    print()
# Generator for files of type odt, doc, etc.
def get_documents(path):
    for curr_path,dirs,files in os.walk(path):
        for f in files:
            try:
                if f.rsplit('.',1)[1].lower() in FILETYPES:
                    yield curr_path,f
            except:
                pass
# pass the root directory as a parameter
for path,f in get_documents(sys.argv[1]):
    process_file(path,f)
```

You may run the above program as follows, from the desktop:

```
$ ./load_doc_files.py ~/Documents | HADOOP_USER_NAME=fedora \
hdfs dfs -fs hdfs://h-mstr/ -put - doc_files.txt
```

The above program can be easily extended to convert and load PDF files as well. Or another program can be used to append to the existing HDFS file.

The obvious next step is that once the data is in Hadoop, you need to figure out how to use it and what you can do with it.

An example of modified word count

Hadoop Map-Reduce offers a very convenient streaming framework. It takes input lines from an HDFS file and passes them on standard input to one or more mapper programs. The standard output of the mapper programs is routed to the standard input of the reducer programs. The mapper and reducer programs could be in any language.

Each line of a mapper output is expected to be a keyword, value pair separated by a tab. All lines with the same keyword are routed to the same reducer.

A common first example is a word count in a text file.

Each line of a text file is split into words by a mapper. It writes the word as the key with the value 1. The reducer counts the number of times a particular word is received. Since the word is the key, all occurrences of the same word will be routed to the same reducer.

In *doc_files.txt*, each line is actually a file. It is very likely that words will be repeated in a line. So, it is better if the mapper counts the words in a line before writing to *stdout*, as